

Типы данных в API

Краткий справочник тестировщика: какие типы бывают в JSON и что с ними проверять

Зачем тестировщику типы данных

- Типы полей описаны в документации API — это часть контракта
- Большинство багов валидации ловится именно на несоответствии типов: число пришло строкой, дата — в неверном формате, вместо объекта — null
- Проверять не только запросы, но и ответы: сервер должен отвечать ровно теми типами, что задокументированы

Integer — целое число

```
{ "age": 25, "count": 3 }
```

- Только целые значения: 25.5 или "25" (строкой) — не подходит
- Проверять границы из документации: minimum, maximum (возраст 0–120 и т.п.)
- Классический баг: сервер возвращает число как строку "age": "25"

String — строка

```
{ "name": "John Doe" }
```

- Проверять ограничения из документации: допустимые символы, длина, pattern (регулярка)
- Пробелы по краям (тримминг), пустая строка "", спецсимволы, кириллица/латиница
- Валидация на входе и на выходе может отличаться — проверяй обе

Boolean — true / false

```
{ "is_active": true }
```

- Только true или false — без кавычек: "true" строкой — это баг
- Не путать с числами: 0 и 1 вместо false и true — частое расхождение

Enum — перечисление

```
{ "status": "candidate" } // только из списка: minor, candidate, retired
```

- Принимаются только значения из списка; всё остальное — 400 Bad Request
- Проверь регистр: "CANDIDATE" не должно пройти, если задокументирован нижний
- Проверь весь список: каждое допустимое значение должно обрабатываться

Float — дробное число

```
{ "price": 19.99, "rating": 4.5 }
```

- Дробная часть через точку, не запятую
- Отрицательные значения, ноль, очень большие числа
- Точность: сколько знаков после точки допускает документация

Array — массив

```
{ "items": [1, 2, 3] }
```

- Пустой массив [] — валидный ответ, когда данных нет (не ошибка!)
- Проверь, что элементы массива одного типа и соответствуют схеме
- Сортировка, порядок, лимиты и пагинация — отдельная тема для списков

Object — объект

```
{ "user": { "id": 1, "name": "John" } }
```

- Все обязательные ключи должны присутствовать; лишние поля — повод проверить документацию
- Вложенные объекты: проверяй типы на каждом уровне
- Ключи регистрозависимы: "Name" вместо "name" — это другое поле

Null — отсутствие значения

```
{ "middle_name": null }
```

- Null ≠ 0, Null ≠ "" (пустая строка), Null ≠ [] (пустой массив) — не подменяй при проверке
- Уточни по документации: в каких полях null допустим, а в каких должен быть валидный объект

Дата и время

```
{ "expires_at": "2026-08-23T12:00:00Z" }
```

- Стандарт — ISO 8601: YYYY-MM-DDTHH:MM:SSZ
- Проверяй таймзоны, границы (даты в прошлом/будущем) и точность (с секундами или без)
- Сроки действия (expires_at) — любимое место багов: протухшая запись всё ещё active

Коды и идентификаторы

```
{ "pass_code": "SCR-A1B2C3D4", "id": 42 }
```

- Формат кодов обычно строго задан (префикс + символы) — сверяй с документацией
- ID — почти всегда целое число; проверяй отрицательные, нулевые и очень большие значения

Типы данных — часть контракта API

Документация описывает не только методы, но и типы каждого поля. Любое отклонение — баг.

Больше полезного про тестирование — в телеграм-канале [@eddytester](#).